Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

Szoftverfejlesztő és -tesztelő

Vizsgaremek

NutriLife – Egészséges életmód alkalmazás

Készítette: Cornuta Dávid

Sinkó Dávid

2026.

Tartalomjegyzék

1. Bevezetés....4
1.1 Témaválasztás indoklása....4
1.2 Kitűzött célok....4
2. Csapatmunka megvalósítása....5
2.1 A csapat működése....5
2.2 Kommunikáció és együttműködés....5
2.3 Csapatfeladatok felosztása....5
Frontend fejlesztő (Sinkó Dávid)....6
Backend fejlesztő (Cornuta Dávid)....6
3. Felhasználói dokumentáció....7
3.1 Az alkalmazás telepítése....7
3.1.1 Backend telepítése és futtatása....7
3.1.2 Frontend telepítése és futtatása....7
3.2 Az alkalmazás használata....8
3.2.1 Nyitó oldal....8
3.2.2 Regisztráció és bejelentkezés....8
3.2.3 Irányítópult (Dashboard)....9
3.2.4 Receptek böngészése....12
3.3.5 Recept részletező oldal....12
3.2.6 Recept feltöltése....13
3.2.7 Étrendterv (Meal Plan)....14
3.2.8 Statisztikák....14
3.2.9 Profil oldal....15
3.2.10 Admin felület....15
4. Fejlesztői dokumentáció....15
4.1 Választott fejlesztői környezetek....15
4.2 Alkalmazott technológiák....16
4.2.1 Frontend technológiák....16
4.2.2 Backend technológiák....16
4.2.2.1 C# és ASP.NET Core....17
4.2.2.2 Entity Framework Core....17
4.2.2.3 SQLite....17
4.2.2.4 JWT hitelesítés....17
4.2.2.5 ImageKit.io....18
4.3 Rendszerarchitektúra....18
4.3.1 Architektúra áttekintése....18
4.3.2 Frontend architektúra....18
4.3.3 Backend architektúra....18
4.3.4 Kommunikáció és biztonság....19
5. Adatbázis modellek....19

5.1 User....19
5.2 Activity / UserActivity....19
5.3 Ingredient / Recipe / RecipeIngredient....20
5.4 Meal....20
5.5 UserAttributes / UserGoal....20
5.6 WeeklyMealPlan / DailyMealPlan....21
5.7 RefreshToken....21
6. Autentikáció és Jogosultságkezelés....22
6.1 Token-alapú hitelesítés....22
6.2 Szerepkörök....22
6.3 Soft Delete....22
7. API Végpontok....22
7.1 Autentikáció - /api/auth....22
7.2 Felhasználói profil - /api/users/me....23
7.3 Testmérés, Cél - /api/users/me/attributes és /goal....23
7.4 Étkezések - /api/users/me/meals....24
7.5 Tevékenységek - /api/users/me/activities....24
7.6 Tevékenységtípusok - /api/activity....24
7.7 Hozzávalók - /api/ingredient....24
7.8 Receptek - /api/recipe....25
8. Admin végpontok....25
8.1 Felhasználók - /api/admin/users....25
8.2 Tevékenységtípusok - /api/admin/activities....26
8.3 Hozzávalók - /api/admin/ingredient....26
8.4 Receptek - /api/admin/recipe....26
9. Tesztelés....26
9.1 Frontend tesztelése....27
9.1.1 signUpValidators – Regisztrációs validációk....27
9.1.2. useAdminFilter – Admin szűrő hook....27
9.1.3 useEditProfileForm – Profilszerkesztő form hook....27
9.1.4 useRecipeFilter – Recept szűrő hook....28
9.1.5 validateRecipePayload – Recept payload validáció....28
9.2 Backend tesztelése....28
9.2.2 Swagger UI....28
9.2.2 Bejelentkezést igénylő végpontok tesztelése....29
10. Továbbfejlesztési lehetőségek....30
10.1 Rövidtávú továbbfejlesztési lehetőségek....30
10.1.1 Bevásárlólista generálása....30
10.1.3 Közösségi funkciók (Kommentek, kedvelések, értékelések)....30
10.2 Hosszabb távú továbbfejlesztési lehetőségek....31
10.2.1 Gamifikáció....31
10.2.2 Értesítések, emlékeztetők....31
11. Irodalomjegyzék....31

1. Bevezetés

1.1 Témaválasztás indoklása

A mai rohanó életmódban az egészséges táplálkozás nyomon követése egyre fontosabbá válik. Számos egészségügyi alkalmazás létezik a piacon, azonban ezek többsége vagy túl bonyolult, vagy nem nyújt elég személyre szabott megoldást. A meglévő alkalmazások elsősorban kalóriaszámlálásra koncentrálnak, azonban receptajánlást és automatikus étrendtervezést jellemzően nem, vagy csak korlátozottan kínálnak.

A NutriLife projekt célja egy olyan modern, webalapú egészségalkalmazás megvalósítása, amely integrálja a kalóriakövetést, a receptböngészést, a heti étrendtervezést és a mozgáskövetést egyetlen, könnyen kezelhető felületen. Az alkalmazás egyedi jellemzője, hogy saját receptadatbázissal rendelkezik, amelyből a rendszer személyre szabott, heti étrendet generál a felhasználó számára – minden napra reggeli, leves, főétel és vacsora ajánlattal. Ez lehetővé teszi, hogy a felhasználó pontosabban és kényelmesebben kövesse nyomon kalória- és tápanyagbevitelét.

1.2 Kitűzött célok

A fejlesztés során az alábbi főbb célokat tűztük ki magunk elé:

- Egy reszponzív, modern felhasználói felület készítése
- Kalóriabevitel és elégetett kalória napi szintű nyomon követése
- Receptadatbázis böngészése részletes szűrési lehetőségekkel
- Automatikusan generált heti étrendterv megjelenítése
- Edzésnaplózás és aktivitáskövetés
- Közösségi recept feltöltési lehetőség
- Statisztikák és grafikonok megjelenítése a felhasználó haladásáról

2. Csapatmunka megvalósítása

2.1 A csapat működése

A fejlesztőcsapat két főből áll. A projekt kezdetén közösen fektettük le az alapokat: meghatároztuk az alkalmazás funkcióit, kialakítottuk az adatbázis-modellt, és elkészítettük a tervezett oldalak vázlatait (wireframe-eket). Ez a kezdeti fázis rendkívül fontosnak bizonyult, mivel elkerülhetővé tette a félreértésekből adódó párhuzamos munkát és a felesleges újrafejlesztést.

A csapat agilis elveket követett a fejlesztés során: rövid, egy-két hetes iterációkban gondolkodtunk, és minden iteráció végén összegeztük az elért eredményeket, azonosítottuk az akadályokat, és meghatároztuk a következő sprint feladatait. Ez a munkamódszer rugalmasságot biztosított, és lehetővé tette a prioritások menet közbeni módosítását.

2.2 Kommunikáció és együttműködés

A csapattagok kommunikációját elsősorban tantermi környezetben valósítottuk meg. A napi egyeztetések az iskolában zajlottak, ahol rendszeren megbeszéltük a teendőket, hogy mindenki naprakész legyen a fejlesztés állapotával kapcsolatban.

A verziókezelésre a Git rendszert és a GitHub platformot használtuk. A fejlesztők két külön github könyvtárba (repository) dolgoztak az egyes funkciókon.

2.3 Csapatfeladatok felosztása

A munkamegosztás nagyjából a frontend és a backend fejlesztés mentén szerveződött, ugyanakkor a csapattagok rendszeresen segítették egymást, és az összekapcsolási pontokat (API integráció, hitelesítés) közösen oldottuk meg.

Frontend fejlesztő (Sinkó Dávid)

Az alkalmazás teljes frontend részének megvalósítása egyetlen csapattag feladata volt. Ő felelt a React-alapú felhasználói felület (UI) teljes körű kialakításáért és az összetett frontend logika megvalósításáért. Munkája során megtervezte és implementálta a reszponzív oldalakat, kialakította a navigációs rendszert, valamint elkészítette az összes oldalkomponenst.

A React Router segítségével megvalósította az egyoldalas alkalmazás útvonalkezelését, beleértve az autentikációhoz kötött útvonalak védelmét és a navigáció animált átmeneteit is. Emellett ő dolgozta ki a receptszűrő rendszert, a statisztikai oldalakat diagramokkal, valamint az alkalmazás állapotkezeléséhez szükséges összetett hook-okat.

Az ő munkájának köszönhető az alkalmazás vizuálisan egységes megjelenése, amely a TailwindCSS és a Material UI együttes használatán alapul. Továbbá integrálta a Framer Motion animációkat is, amelyek gördülékenyebb és életszerűbb felhasználói élményt biztosítanak.

Backend fejlesztő (Cornuta Dávid)

A második csapattag volt felelős a projekt teljes ASP.NET Core alapú REST API backendjének megvalósításáért, valamint az Adminisztrátori felület backend és frontend logikájának megtervezéséért. Feladatai közé tartozott az adatbázis-modell (Entity Framework Core) megtervezése és implementálása, a JWT-alapú hitelesítési rendszer kiépítése refresh token rotációval, valamint az összes API végpont elkészítése. Ő valósította meg a soft delete mechanizmust, a kulcsfontosságú szolgáltatásokat (CaloriesCalculationService és WeeklyMealService), továbbá az ImageKit.io integrációt a képkezeléshez. Az adminisztrátori felület (felhasználók, hozzávalók, tevékenységek és receptek kezelése), a backend logikája és az ehhez szükséges API végpontok megvalósítása az ő felelőssége volt.

3. Felhasználói dokumentáció

3.1 Az alkalmazás telepítése

Klónozd le a repository-t:

```
git clone https://github.com/DavidC112/Vizsgaremek-2025-26.git
```

3.1.1 Backend telepítése és futtatása

A backend alkalmazás futtatásához szükség van az alábbi előfeltételekre:

- .NET 10 SDK

1. Nyisd meg a backend projektet Visual Studio Code-ban
2. A terminálban futtasd a backendet:

```
dotnet build
dotnet run
```

A backend alapértelmezetten a http://localhost:5140 címen érhető el.

Swagger UI elérhető a http://localhost:5140/swagger

3.1.2 Frontend telepítése és futtatása

A frontend alkalmazás futtatásához az alábbi előfeltételek szükségesek:

- Node.js 18 vagy újabb verzió
- npm (Node Package Manager) – Node.js-sel együtt települ
- Git verziókezelő rendszer

Az alkalmazás telepítési lépései a következők:

1. Nyisd meg a frontend mappát egy új Visual Studio Code ablakban:

2. Telepítsd a függőségeket

```
npm install
```

3. Indítsd el az alkalmazást

```
npm install
```

A frontend alapértelmezetten a http://localhost:5173 címen érhető el

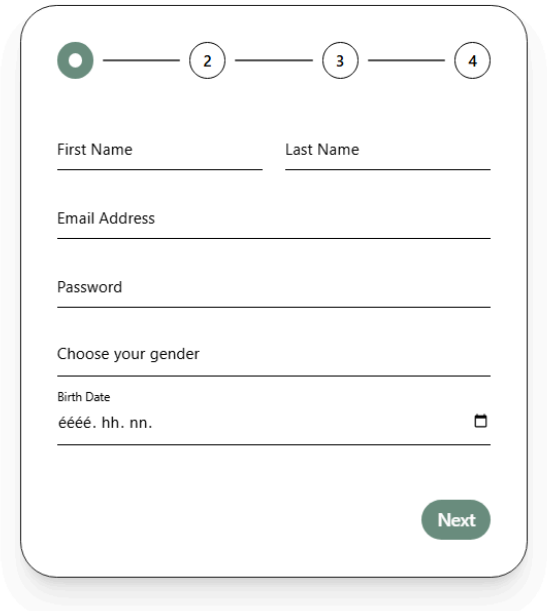
3.2 Az alkalmazás használata

3.2.1 Nyitó oldal

A Nyitó oldal (Landing Page) az alkalmazás bevezető felülete, amelynek elsődleges célja, hogy a még nem regisztrált vagy nem bejelentkezett felhasználók számára átfogó összegzést nyújtson a NutriLife alkalmazásról. Az oldalon elhelyezett gombok biztosítják a zökkenőmentes átirányítást a Bejelentkezési (Login) vagy a Regisztrációs (Sign up) oldalra.

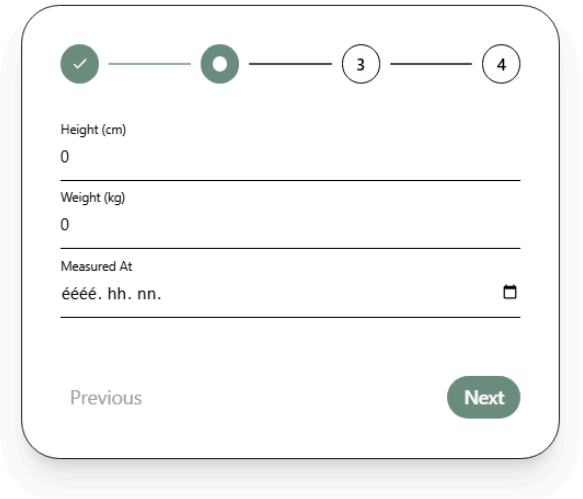
3.2.2 Regisztráció és bejelentkezés

Új felhasználó létrehozásakor egy háromoldalas, lépésenként haladó (Stepper) regisztrációs folyamat segíti a felhasználót:

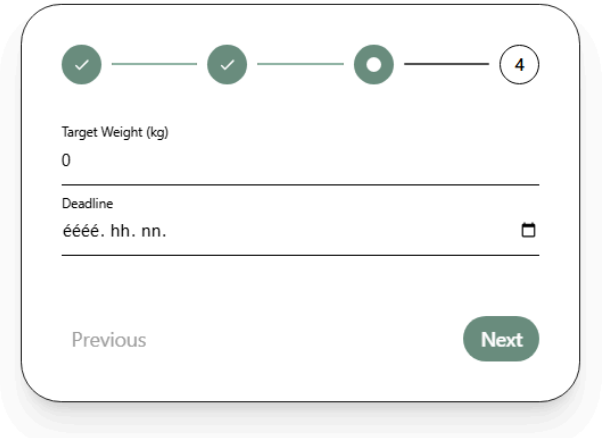


1. lépés: Alapadatok megadása (név, e-mail, jelszó, nem, születési dátum)

A jelszónak meg kell felelnie a biztonsági követelményeknek: legalább 8 karakter, kis- és nagybetű, szám és speciális karakter szükséges.



2. lépés: Testméretek megadása (magasság, testtömeg, mérés dátuma)



3. lépés: Célok meghatározása (célsúly, határidő)

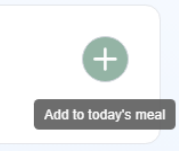
A cél meghatározása (Mit szeretne elérni a felhasználó): A felhasználó által megadott aktuális testtömeg és a kitűzött célsúly (TargetWeight) alapján az alkalmazás eldönti, hogy a felhasználó célja súlygyarapodás, súlycsökkenés vagy súlytartás.

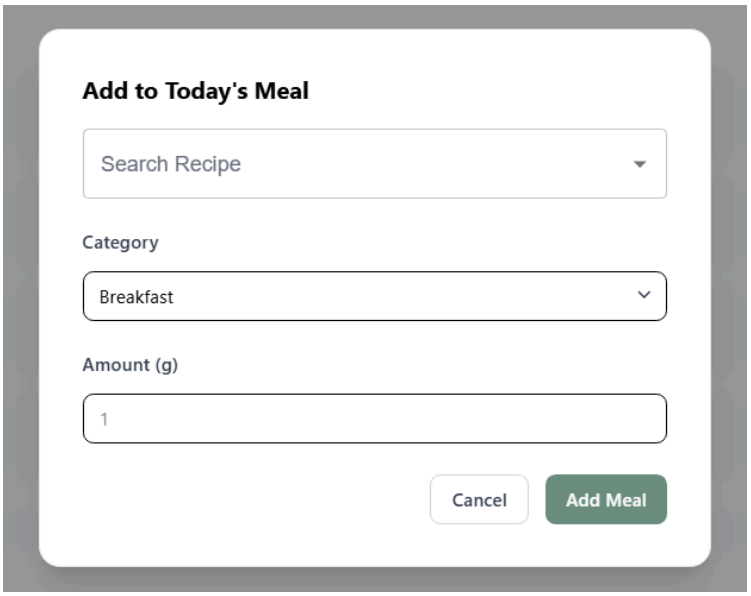
3.2.3 Irányítópult (Dashboard)

Az irányítópult az alkalmazás központi felülete, amelyre minden bejelentkezés után érkezik a felhasználó. Az oldal felső részén a Napi kalóriacél kártya látható, amely animált számokkal és egy haladásjelzővel jeleníti meg az aktuális kalóriamérleget. A kártya tartalmazza a fogyasztott, az elégetett és a nettó kalóriák értékét, valamint a napi cél százalékos teljesítettségét.

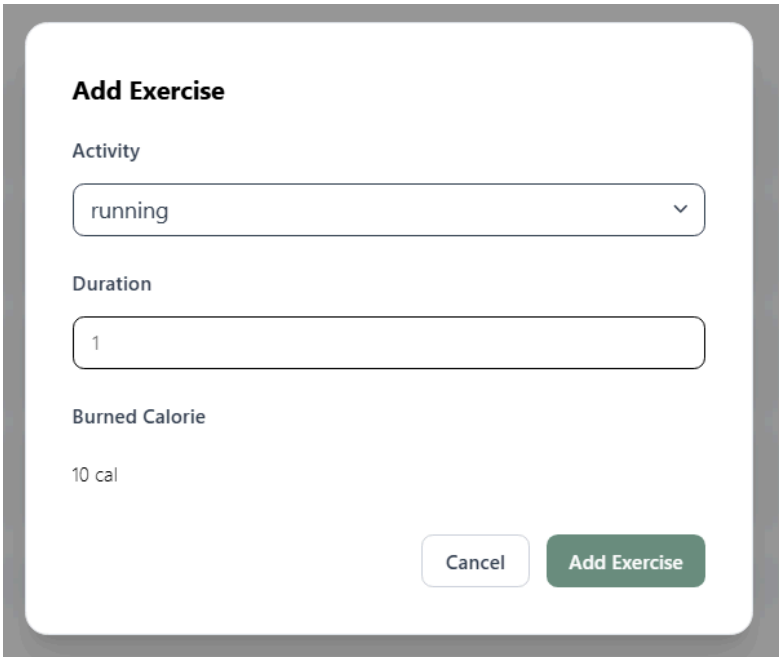
Alatta az Ajánlott étkezések szekció jelenik meg, amely a heti étrendterv mai napra vonatkozó részét mutatja: reggelit, levest, ebédet és vacsorát, mindegyikhez navigációs gombot biztosítva az adott recept részletes oldalára.

A Mai étkezések szekció a felhasználó által ténylegesen naplózott ételeket listázza, az egyes tételek kalória- és makrotápanyag-értékeivel együtt. Új ételt a jobb felső sarokban lévő plusz gombra kattintva lehet felvenni. Rákattintva előugrik egy ablak amelyben lehet keresni recept nevére, meg lehet adni a recept kategóriát és hogy mennyit fogyasztott grammra

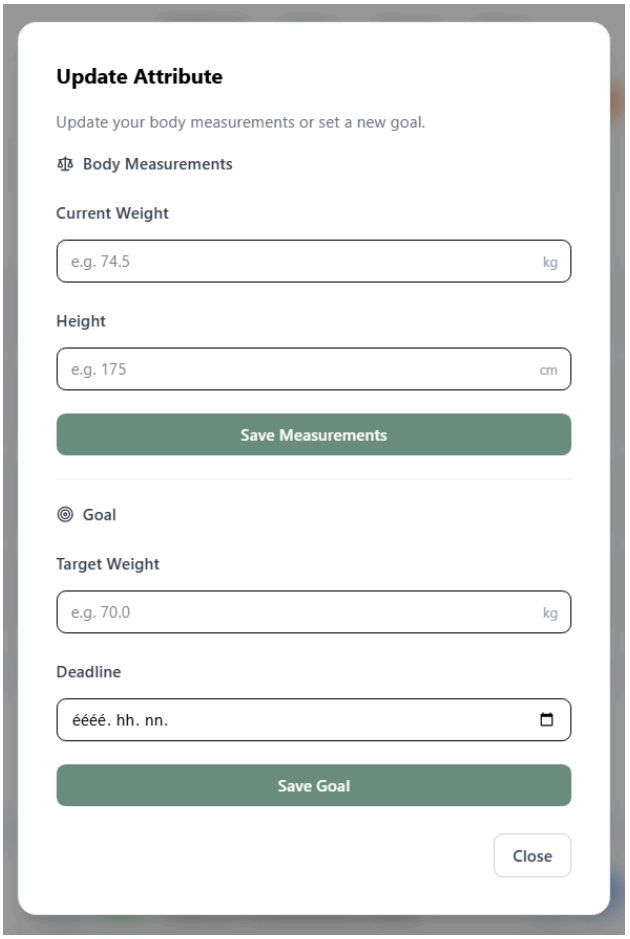




Az oldal alsó részén három gomb található: az első az edzésnaplózó modált nyitja meg, a második a testmérési adatok frissítésére szolgáló modált, a harmadik pedig a receptböngészőre navigál.

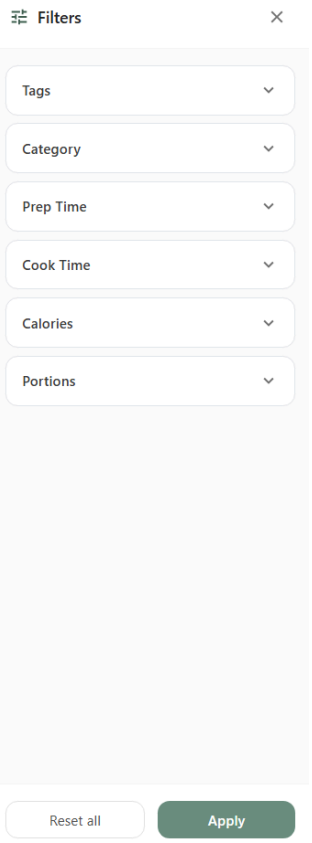


Az edzés hozzáadás ablakban a felhasználónak meg kell adnia hogy milyen tevékenységet végzett, és hogy hány percig. Az adatokból a rendszer kiszámolja hogy mennyi kalóriát égetett a felhasználó.



Az Update Attribute gombra kattintva előjön egy modal, amelyben lehetőség van frissíteni a felhasználónak a súlyát és magasságát vagy egy új célt állítására magának.

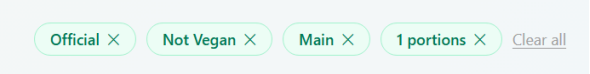
3.2.4 Receptek böngészése



A Receptek oldal egy kártyás elrendezésben jeleníti meg az összes elérhető receptet. A részletes szűrési lehetőségek egy oldalsó fiókban (Drawer) találhatók:

- Szöveges keresés a recept neve alapján
- Típus szerinti szűrés: közösségi / hivatalos, vegán, vegetáriánus
- Kategória: Reggeli, Leves, Főétel
- Időtartam szűrők: előkészítési és főzési idő csúszkával
- Kalóriatartomány szűrő csúszkával (0–2000 kcal)
- Adagszám szűrő

Az aktív szűrők chip-ként jelennek meg az oldal tetején, egyenként vagy egyszerre törölhetők. A receptkártyán megtalálható a recept neve, kategória-jelölők, rövid leírás, kalóriaérték, elkészítési idő, adagszám és makrotápanyagok.

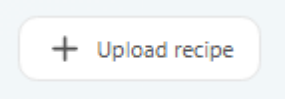


3.3.5 Recept részletező oldal

Egy receptre kattintva megnyílik annak részletes oldala, amely tartalmaz:

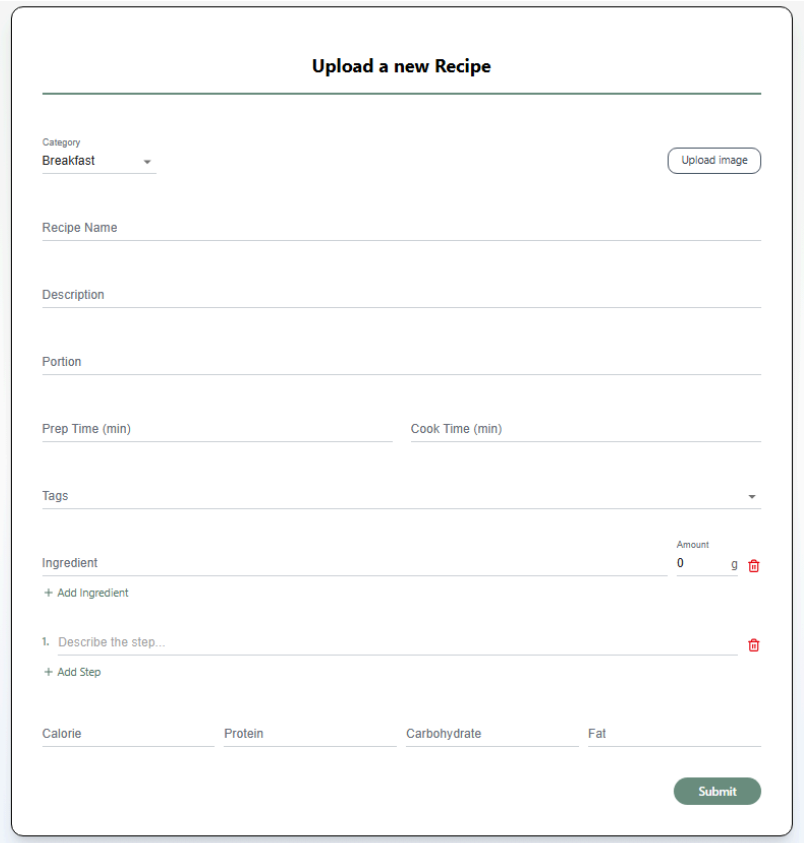
- Recept neve és szerzője (közösségi recept esetén a feltöltő neve)
- Étrendi jelölők (vegán, vegetáriánus, közösségi)
- Receptfotó
- Rövid leírás
- Előkészítési és főzési idő, adagszám
- Részletes tápanyagtáblázat (100g-ra vonatkoztatva)
- Összetevők listája grammban megadva
- Elkészítési lépések számozva
- Gomb a recept mai étkezéshez adásához (kategória és mennyiség megadásával)

3.2.6 Recept feltöltése



A Receptek oldalon belül lehetőség van új receptet hozzáadni a felhasználónak. Az Upload recipe gombra kattintva átirányít az alkalmazás a receptfeltöltő szekcióba.
A NutriLife alkalmazásban egy közösségi recept feltöltéséhez (létrehozásához) a következő adatokat kell megadni az API hívás (payload) és a validáció alapján:

- Recept adatok:
  - Név
  - Kategória
  - Leírás
  - Elkészítési lépések
  - Adagszám
- Idő adatok:
  - Előkészítési idő
  - Főzési idő
- Étrendi jelölők (tagek):
  - Vegán
  - Vegetáriánus
  - (Megjegyzés: A `isCommunity` jelölő is a recept része, de ez a közösségi recept létrehozásakor automatikusan beállításra kerül).
- Hozzávalók:
  - A hozzávalók listája, amely legalább egy elemet tartalmaz, és minden hozzávalónak meg kell adni az azonosítóját és a mennyiségét (grammban megadva).
- Tápanyag adatok:
  - A recepthez szükségesek a kalória, fehérje, szénhidrát és zsír adatok.
- Kép:
  - A recepthez utólag külön API végponton keresztül lehet feltölteni a receptfotót (multipart/form-data formátumban).



3.2.7 Étrendterv (Meal Plan)

A Meal Plan oldalon a teljes heti étrendterv megtekinthető. A napok között animált gombokkal lehet navigálni. Minden naphoz négy étkezési kategória (reggeli, leves, főétel, vacsora) jelenik meg a receptkártyák stílusában, megfelelő színkódolt jelölőkkel.

3.2.8 Statisztikák

A Statisztikák oldal három fő szekcióból áll:

- Összefoglaló kártyák: Átlagosan elfogyasztott és elégetett kalória
- Grafikonok: Súlyváltozás vonaldiagram időbeli bontásban; makrotápanyag-megoszlás félkör-diagram (szénhidrát, fehérje, zsír)
- Részletes napló: Lapon belüli fülek az étkezési napló és az aktivitás napló között, összecsukható napi bontással

3.2.9 Profil oldal

A profil oldalon a felhasználó megtekintheti és szerkesztheti személyes adatait, feltöltött receptjeit.
Lehetőség van szerkeszteni:

- Név
- E-mail
- Születési dátum
- Jelszó

Ezen kívül a fiók törlésére is van lehetőség.

A feltöltött receptek kártyás formában jelennek meg, amelyeken szerkesztési és törlési lehetőség is elérhető.

3.2.10 Admin felület

Adminisztrátor jogosultságú felhasználók számára elérhető az /admin útvonal, amely négy aloldalt tartalmaz:

- Felhasználók: Minden regisztrált felhasználó kezelése, törlés és visszaállítás, részletek megtekintése
- Aktivitások: Sportaktivitások hozzáadása, szerkesztése, törlése és visszaállítása
- Összetevők: Élelmiszer-összetevők adatainak kezelése
- Receptek: Az összes recept adminisztrálása képfeltöltéssel, szerkesztéssel és moderálással

Az admin felület minden listanézetében elérhető szöveges keresés és szűrési lehetőség (pl. csak törölt elemek megjelenítése).

4. Fejlesztői dokumentáció

4.1 Választott fejlesztői környezetek

A fejlesztés során az alábbi eszközöket és környezeteket alkalmaztuk:

Technológia	Leírás / Felhasználás
Kódszerkesztő	Visual Studio Code – kiterjesztésekkel (ESLint, Prettier, Tailwind CSS IntelliSense), Visual Studio
Verziókezelés	Git és GitHub
Böngészőeszközök	Chrome DevTools, React Developer Tools böngészőbővítmény

API tesztelés	Swagger UI a backend dokumentációjához, böngésző DevTools Network fül
Terminál	Windows Terminal / bash

4.2 Alkalmazott technológiák

4.2.1 Frontend technológiák

A felhasználói felület egyoldalas alkalmazásként (SPA – Single Page Application) lett megvalósítva, a React könyvtár 19-es verziójával. A TypeScript statikus típusozás bevezetése komoly minőségbiztosítási előnyt jelent: a típushibák fordítási időben derülnek ki, és a kódszerkesztő intelligens kódkiegészítést biztosít.

Technológia	Leírás / Felhasználás
React 19	A felhasználói felület építőköve; funkcionális komponensek, hookok
TypeScript	Statikus típusozás a fejlesztési hibák korai felderítéséhez
Vite 7	Gyors fejlesztési szerver és optimalizált build eszköz
Tailwind CSS 4	Utility-first CSS keretrendszer reszponzív dizájnhoz
React Router 7	Kliensoldali útválasztás, védett útvonalakkal
Axios	HTTP kliens az API kommunikációhoz, interceptorral
Framer Motion	Animációk és átmenetek a felhasználói felületen
Material UI 7	UI komponenskönyvtár (Autocomplete, Drawer, Accordion, Slider, Charts)
Headless UI	Akadálymentes, stilizálható modal és form komponensek
Lucide React	Ikon könyvtár
React Three Fiber	WebGL alapú 3D selyemhatású animáció a bejelentkezési oldalon

4.2.2 Backend technológiák

A szerver oldal az ASP.NET Core 10 keretrendszerre épül, C# programozási nyelvvel. Az adatbázis-kezelés Entity Framework Core ORM-mel, Code-First megközelítéssel történik, SQLite adatbázissal. A hitelesítési rendszer ASP.NET Core Identity + JWT Bearer tokenekre épül, refresh token rotációval.

A képek tárolása a külső ImageKit.io CDN szolgáltatáson keresztül valósul meg.

Technológia	Leírás / Felhasználás
ASP .NET Core 10	Backend keretrendszer
Entity Framework Core	ORM (Object-Relational Mapping, azaz objektum-relációs leképzés)
SQLite	Állományalapú relációs adatbázis-kezelő rendszer
ASP.NET Core Identity + JWT Bearer	Autentikáció, hitelesítés
ImageKit.io	A képek tárolásához használt CDN

4.2.2.1 C# és ASP.NET Core

Az ASP.NET Core egy nyílt forráskódú keretrendszer, amely a következő okokból ideális választás:

- Erősen típusos C# nyelv - a hibák nagy részét fordítási időben kiszűri, növeli a kód megbízhatóságát.
- Aktív fejlesztés, LTS támogatás - rendszeres biztonsági frissítésekkel.
- Platformfüggetlenség - Windows, Linux és macOS rendszeren egyaránt futtatható.

4.2.2.2 Entity Framework Core

Az EF Core egy ORM (Object-Relational Mapping) eszköz, amely C# osztályokból automatikusan generálja az adatbázissémát (Code-First). Előnyei:

- LINQ alapú lekérdezések - SQL injection elleni védelem, olvasható szintaxis.
- Migrációkezelés - az adatmodell változásai verziókövethető migrációs fájlokban rögzülnek.

4.2.2.3 SQLite

Szerver nélküli, fájlalapú relációs adatbázis. Fejlesztési fázisban nulla konfigurációval, egyetlen .db fájlban tárolja az összes adatot. Könnyen verziókezelhető, EF Core-ral natívan kompatibilis.

4.2.2.4 JWT hitelesítés

Az ASP.NET Core Identity kezeli a jelszó-hashelést és a felhasználókezelést. A végpontok védelmét JWT Bearer tokenek biztosítják, refresh token rotációval. Ez a standard, jól skálázható megoldás.

4.2.2.5 ImageKit.io

A profilképek és receptképek tárolása külső CDN-en történik, ami tehermentesíti a szervert a bináris fájlok kezelése alól és gyors képkiszolgálást biztosít.

4.3 Rendszerarchitektúra

4.3.1 Architektúra áttekintése

Az alkalmazás klasszikus kliens-szerver architektúrát követ, amelyen belül három jól elkülönített réteg azonosítható: a frontend SPA (böngészőben fut), a RESTful backend API (szerveren fut), és az adatbázis (SQLite fájl, amelyet az ORM kezel). A frontend és a backend kizárólag HTTP(S) alapú REST API hívásokkal kommunikálnak egymással – ez a lazán csatolt (loosely coupled) megközelítés lehetővé teszi, hogy a két rész egymástól függetlenül fejlődjön, és akár különböző infrastruktúrán üzemeljen.

4.3.2 Frontend architektúra

A frontend React alkalmazásban a Context API és az egyéni hook-ok (custom hooks) biztosítják az állapotkezelést és az adatok beolvasását. Az Axios HTTP kliens request interceptorral van felszerelve, amely automatikusan mellékeli a JWT access tokent minden hitelesített kérés fejlécébe.

Ha a szerver 401 Unauthorized választ küld vissza (lejárt token), az Axios response interceptor automatikusan megpróbálja megújítani a tokent a /api/auth/refresh végpont meghívásával, és az eredeti kérést megismétli.

4.3.3 Backend architektúra

A backend ASP.NET Core alkalmazás layered (rétegelt) architektúrát követ:

- a kontrollerek fogadják az HTTP kéréseket és adnak vissza válaszokat
- a service osztályok (CaloriesCalculationService, WeeklyMealService, DailyIntakeService) tartalmazzák a logikai számításokat

- Összetevők: Élelmiszer-összetevők adatainak kezelése
- az Entity Framework DbContext pedig az adatbázis-műveleteket végzi

Ez a szétválasztás megkönnyíti a tesztelhetőséget és a kód karbantarthatóságát.

4.3.4 Kommunikáció és biztonság

Az alkalmazás CORS (Cross-Origin Resource Sharing) szabályokkal van felszerelve, amelyek csak a frontend fejlesztői portját (https://localhost:5173) engedélyezik. Ez egyrészt fejlesztési biztonságot nyújt, másrészt könnyen konfigurálható éles környezetre is.

5. Adatbázis modellek

Az összes modell C# osztályként van definiálva (Code-First EF Core). Az IDeletable interfészt implementáló modellek soft delete-et támogatnak (IsDeleted mező).

5.1 User

Az IdentityUser-ből örököl. Saját mezők: FirstName, LastName, BirthDate, Age [NotMapped], Gender, CreatedAt, IsDeleted, ProfilePictureUrl, FileId. Kapcsolatok: UserAttributes (1:N), UserActivity (1:N), UserGoal (1:1), Meal (1:N), WeeklyMealPlan (1:1), Recipe (1:N).

5.2 Activity / UserActivity

Az Activity tábla az adminisztrátor által felvett tevékenységtípusokat tárolja (Name, CaloriesBurnedPerHour, IsDeleted). A UserActivity a felhasználó konkrét edzésnaplóját rögzíti: ActivityId (FK), Duration (perc), CaloriesBurned [NotMapped] = CaloriesBurnedPerHour × Duration / 60, Log (dátum).

5.3 Ingredient / Recipe / RecipeIngredient

Az Ingredient hozzávalókat és a hozzájuk tartozó tápanyagértékeket tárolja 100g-ra (Name, Calories, Protein, Carbohydrate, Fat, IsDeleted). A Recipe tartalmazza a recept adatait (Name, Category, Description, Instructions, Portion, makrotápanyagok, ImageUrl, FileId (kép törléséhez), IsCommunity, IsVegan, IsVegetarian, IsDeleted). A RecipeIngredient kapcsolótábla köti össze a kettőt a hozzávaló mennyiségével (N:M, PK: RecipeId + IngredientId).

5.4 Meal

A naplózott étkezések táblája. Hivatkozhat receptre (RecipeId) VAGY hozzávalóra (IngredientId), tárolja az Amount (gramm) értéket. A CalculateNutrition() metódus a bevitt mennyiség arányában számítja ki a tényleges tápanyagtartalmat.

```
public NutritionDto CalculateNutrition()
{
    var source :object? = (object?)Recipe ?? Ingredient;

    if (source == null)
        return new NutritionDto();

    return source switch
    {
        Recipe recipe => new NutritionDto
        {
            Calories = recipe.Calories * Amount / 100m,
            Protein = recipe.Protein * Amount / 100m,
            Fat = recipe.Fat * Amount / 100m,
            Carbohydrate = recipe.Carbohydrate * Amount / 100m
        },

        Ingredient ingredient => new NutritionDto
        {
            Calories = ingredient.Calories * Amount / 100m,
            Protein = ingredient.Protein * Amount / 100m,
            Fat = ingredient.Fat * Amount / 100m,
            Carbohydrate = ingredient.Carbohydrate * Amount / 100m
        },

        _ => new NutritionDto()
    };
}
```

5.5 UserAttributes / UserGoal

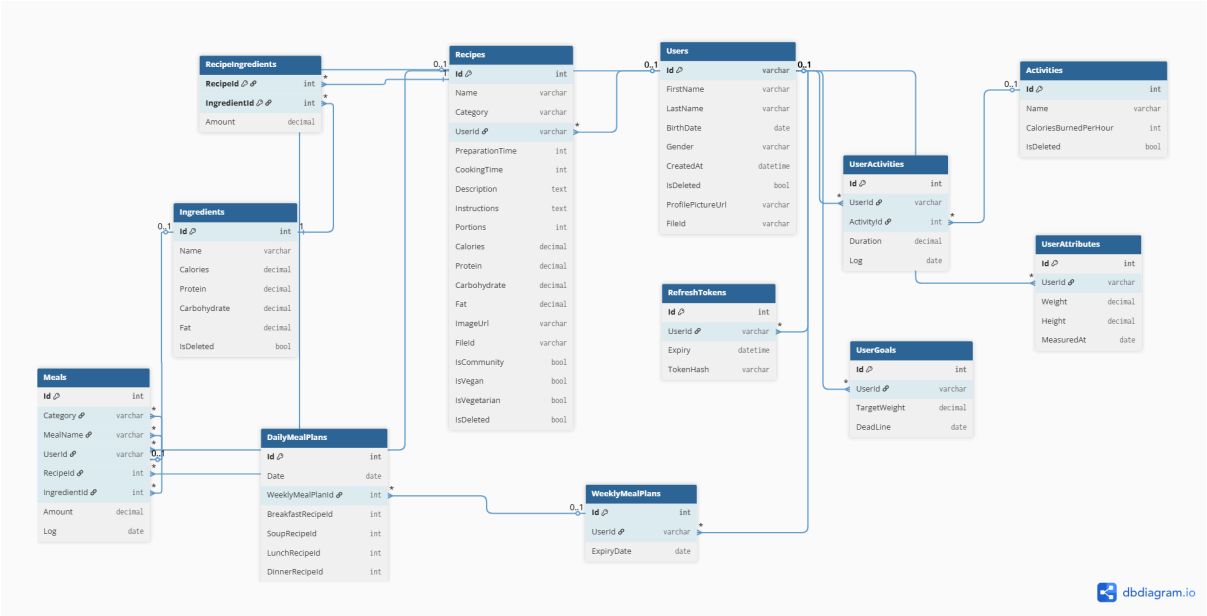
UserAttributes: időbélyeges testmérési bejegyzések (Weight, Height, BMI [NotMapped]). UserGoal: a felhasználó célsúlya és határideje (TargetWeight, DeadLine).

5.6 WeeklyMealPlan / DailyMealPlan

A WeeklyMealPlan egy felhasználóhoz kapcsolódó heti étrendterv (lejárati dátum: ExpiryDate), amely hét DailyMealPlan bejegyzést tartalmaz. Minden DailyMealPlan egy naphoz tartozik, és négy étkezést ír le receptazonosítókkal: Breakfast, Soup, Lunch, Dinner (+ RecipeId-k).

5.7 RefreshToken

A JWT refresh token rotációhoz szükséges. Tárolja a TokenHash, UserId és Expiry (lejárat dátuma) mezőket. Bejelentkezéskor keletkezik, kijelentkezéskor vagy megújításkor törlődik.



Entitás A	Kapcs.	Entitás B	Megjegyzés
User	1:N	UserAttributes / Meal / UserActivity / Recipe	Egy felhasználóhoz több bejegyzés tartozhat
User	1:1	UserGoal / WeeklyMealPlan	Egy aktív cél és egy aktuális heti terv
WeeklyMealPlan	1:N	DailyMealPlan	7 napi terv tartozik egy heti tervhez
Recipe	N:M	Ingredient	RecipeIngredient kapcsolótáblán keresztül
Meal	N:1	Recipe / Ingredient	Vagy recepthez, vagy hozzávalóhoz kapcsolódik
UserActivity	N:1	Activity	Több napló hivatkozhat ugyanarra a típusra

6. Autentikáció és Jogosultságkezelés

6.1 Token-alapú hitelesítés

Bejelentkezéskor a szerver egy rövid élettartamú JWT access tokent és egy hosszabb élettartamú refresh tokent.

Amikor az access token lejár, a kliens az /api/auth/refresh végponton megújíthatja. Ilyenkor a régi refresh token érvénytelen lesz (rotáció), és új keletkezik. Kijelentkezéskor a refresh token törlődik az adatbázisból.

6.2 Szerepkörök

Szerepkör	Hozzáférés
User (bejelentkezett)	Saját adatok kezelése, receptek böngészése, közösségi receptek létrehozása és saját receptek szerkeztése, étkezések és tevékenységek naplózása
Admin	Teljes User + összes felhasználó, alapanyag, tevékenység és recept adminisztrálása (beleértve a törölt elemek visszaállítását)

6.3 Soft Delete

Az IDeletable interfészt implementáló modellek (User, Activity, Ingredient, Recipe) törléskor csak IsDeleted = true értéket kapnak. Ez lehetővé teszi a visszaállítást és megőrzi a hivatkozási integritást. Az admin végpontokon keresztül a törölt elemek visszaállíthatók.

7. API Végpontok

Minden endpoint JSON formátumot használ. Az [Auth] jelölés bejelentkezett felhasználót igényel. Képfeltöltő végpontoknál a kérés formátuma multipart/form-data.

7.1 Autentikáció - /api/auth

[POST] /api/auth/register

Új fiók létrehozása. Visszaad access + refresh token párt.

```
Kérés: { firstName, lastName, email, password, birthDate, gender }
Válasz: { accessToken, refreshToken }
```

[POST] /api/auth/login

Bejelentkezés e-maillel és jelszóval.

```
Kérés: { email, password }
Válasz: { accessToken, refreshToken }
```

[POST] /api/auth/refresh

Access token megújítása. A régi refresh token invalidálódik.

```
Kérés: { refreshToken }
Válasz: { accessToken, refreshToken (új) }
```

[POST] /api/auth/logout

[Auth] Kijelentkezés - refresh token törlése az adatbázisból.

```
Kérés: { refreshToken }
Válasz: HTTP 200 OK
```

7.2 Felhasználói profil - /api/users/me

[GET] /api/users/me

[Auth] Bejelentkezett felhasználó adatainak lekérdezése.

```
Válasz: { id, firstName, lastName, email, gender, birthDate, age, profilePictureUrl }
```

[POST] /api/users/me/upload-picture

[Auth] Profilkép feltöltése.

```
Kérés: file
Válasz: { profilePictureUrl }
```

[PATCH] /api/users/me/edit

[Auth] Felhasználói profil szerkesztése (név, email, születési év)

```
Kérés: { firstName, lastName, email, birthDate }
Válasz: HTTP 200 OK
```

[PATCH] /api/users/me/change-password

[Auth] Jelszó változtatása

```
Kérés: { currentPassword, newPassword }
Válasz: HTTP 200 OK
```

[GET] /api/users/me/daily-intake

[Auth] Az aktuális nap összesített tápanyagbevitele.

```
Válasz: { date, calories, protein, carbohydrate, fat }
```

[GET] /api/users/me/weekly-meal-plan

[Auth] Aktuális heti étrendterv lekérdezése. Ha lejárt vagy nem létezik, automatikusan generálódik egy új.

```
Válasz: { expiryDate, dailyMeals: [{ date, breakfast, soup, lunch, dinner, RecipeId-k }] }
```

[DELETE] /api/users/me/delete-picture

[Auth] Profilkép törlése, visszaáll az alapértelmezett képre.

```
Válasz: HTTP 200 OK
```

[PATCH] /api/users/me/delete-account

[Auth] Saját fiók soft-delete törlése.

```
Válasz: HTTP 200 OK
```

7.3 Testmérés, Cél - /api/users/me/attributes és /goal

[GET] /api/users/me/attributes

[Auth] Összes testmérési bejegyzés lekérdezése időrendben.

```
Válasz: [{ id, weight, height, bmi, measuredAt }]
```

[POST] /api/users/me/attributes/add

[Auth] Új testmérési adat rögzítése.

```
Kérés: { weight, height, measuredAt }
```

Válasz: HTTP 200 OK

[GET] /api/users/me/goal

[Auth] Aktuális cél lekérdezése.
Válasz: { targetWeight, deadLine }

[POST] /api/users/me/goal/add

[Auth] Új cél megadása (felülírja a korábbit).
Kérés: { targetWeight, deadLine }
Válasz: HTTP 200 OK

7.4 Étkezések - /api/users/me/meals

[GET] /api/users/me/meals

[Auth] Az aktuális nap étkezéseinek listája tápanyagértékekkel.
Válasz: [{ id, category, mealName, amount, calories, protein, fat, carbohydrate, log }]

[POST] /api/users/me/meals/add

[Auth] Étkezés naplózása. recipeId VAGY ingredientId szükséges.
Kérés: { category, mealName, recipeId?, ingredientId?, amount }
Válasz: HTTP 200 OK

7.5 Tevékenységek - /api/users/me/activities

[GET] /api/users/me/activities

[Auth] Tevékenységnapló lekérdezése.
Válasz: [{ id, activityName, duration, caloriesBurned, log }]

[POST] /api/users/me/activities/add

[Auth] Új edzésbejegyzés rögzítése.
Kérés: { activityId, duration }
Válasz: HTTP 200 OK

[PATCH] /api/users/me/activities/{id}/edit

[Auth] Meglévő bejegyzés időtartamának módosítása.
Kérés: { duration }
Válasz: HTTP 200 OK

7.6 Tevékenységtípusok - /api/activity

[GET] /api/activity/all

Összes elérhető tevékenységtípus (publikus végpont).
Válasz: [{ id, name, caloriesBurnedPerHour }]

[GET] /api/activity/{id}

Egy adott tevékenységtípus adatai.
Válasz: { id, name, caloriesBurnedPerHour }

7.7 Hozzávalók - /api/ingredient

[GET] /api/ingredient/all

Összes hozzávaló listázása (publikus végpont).
`Válasz: [{ id, name, calories, protein, carbohydrate, fat }]`

[GET] /api/ingredient/{id}

Egy hozzávaló részletes adatai.
`Válasz: { id, name, calories, protein, carbohydrate, fat }`

7.8 Receptek - /api/recipe

[GET] /api/recipe/all

Összes elérhető recept listázása (publikus végpont).
`Válasz: [{ id, name, category, preparationTime, portions, calories, isVegan, isVegetarian, isCommunity, imageUrl }]`

[GET] /api/recipe/{id}

Egy recept teljes adatai a hozzávalókkal együtt.
`Válasz: { ...recept adatok, ingredients: [{ ingredientId, name, amount }] }`

[POST] /api/recipe/community/create

[Auth] Közösségi recept létrehozása.
`Kérés: { name, category, preparationTime, cookingTime, description, instructions, portions, isVegan, isVegetarian, ingredients: [...] }`
`Válasz: { id }`

[POST] /api/recipe/community/create/{id}/upload-image

[Auth] Kép feltöltése a recepthez (multipart/form-data).
`Kérés: file = <képfájl>`
`Válasz: { imageUrl }`

[PATCH] /api/recipe/community/{id}/edit

[Auth] Saját közösségi recept szerkesztése.
`Kérés: { módosítandó mezők }`
`Válasz: HTTP 200 OK`

[PATCH] /api/recipe/community/{id}/delete

[Auth] Saját közösségi recept törlése.
`Válasz: HTTP 200 OK`

8. Admin végpontok

Minden admin végpont [Admin] jogosultságot igényel. Az admin CRUD műveletek mindenhol kiterjednek a soft-delete törlésre és visszaállításra is.

8.1 Felhasználók - /api/admin/users

Metódus	Útvonal	Leírás
GET	/api/admin/users/all	Összes felhasználó listázása (töröltek is)
GET	/api/admin/users/{id}	Egy felhasználó adatai
PATCH	/api/admin/users/{id}/delete	Felhasználó soft-delete törlése
PATCH	/api/admin/users/{id}/restore	Törölt felhasználó visszaállítása

8.2 Tevékenységtípusok - /api/admin/activities

Metódus	Útvonal	Leírás
GET	/api/admin/activities/all	Összes tevékenység (töröltek is)
POST	/api/admin/activities/add	Új típus hozzáadása: { name, caloriesBurnedPerHour }
PATCH	/api/admin/activities/{id}/edit	Tevékenység szerkesztése
PATCH	/api/admin/activities/{id}/delete	Soft-delete törlés
PATCH	/api/admin/activities/{id}/restore	Visszaállítás

8.3 Hozzávalók - /api/admin/ingredient

Metódus	Útvonal	Leírás
GET	/api/admin/ingredient/all	Összes hozzávaló (töröltek is)
POST	/api/admin/ingredient/add	Új hozzávaló: { name, calories, protein, carbohydrate, fat }
PATCH	/api/admin/ingredient/{id}/edit	Hozzávaló szerkesztése
PATCH	/api/admin/ingredient/{id}/delete	Soft-delete törlés
PATCH	/api/admin/ingredient/{id}/restore	Visszaállítás

8.4 Receptek - /api/admin/recipe

Metódus	Útvonal	Leírás
GET	/api/admin/recipe/all	Összes recept (töröltek is)
POST	/api/admin/recipe/create	Új hivatalos recept létrehozása
POST	/api/admin/recipe/create/{id}/upload-image	Kép feltöltése recepthez
PATCH	/api/admin/recipe/{id}/edit	Recept szerkesztése
PATCH	/api/admin/recipe/{id}/delete	Soft-delete törlés
PATCH	/api/admin/recipe/{id}/restore	Visszaállítás

9. Tesztelés

9.1 Frontend tesztelése

9.1.1 signUpValidators – Regisztrációs validációk

Ezek unit tesztek tisztán funkcionális segédfüggvényekre. A következő funkciók vannak tesztelve:

- isNonEmpty – üres/whitespace string detekció
- isEmail – email formátum validáció (trimmelés, aldomain, szóközök)
- isMinLength – minimális hossz ellenőrzés, default érték 6
- isValidIsoDate – ISO dátum validáció, placeholder (yyyy-mm-dd) kezelés
- isStrongPassword / getPasswordIssues – erős jelszó kritériumok (nagy/kisbetű, szám, speciális karakter), hibaüzenet generálás
- validateUserDetails, validateUserAttributes, validateUserGoals – összetett validátorok, amelyek { isValid, errors } objektumot adnak vissza, és képesek több hibát egyszerre jelezni

9.1.2. useAdminFilter – Admin szűrő hook

Custom React hook tesztje renderHook segítségével. A hook általánosan paraméterezhető (searchFn, filterFn, initialFilters), és a következők kerülnek tesztelésre:

- Kezdeti állapot (search = "", filterek az initialFilters-ből)
- Keresés case-insensitive módon, törlés utáni állapot
- Filter váltás (pl. törölt/nem törölt)
- Keresés + filter kombinálva
- `filterFn` nélküli működés
- Üres adattömb kezelése

9.1.3 useEditProfileForm – Profilszerkesztő form hook

Szintén egy custom hook, állapotkezeléssel. Teszteli:

- Kezdeti állapot feltöltése egy User objektumból
- set(field, value) – egyedi mezők frissítése, köztük File objektum és null
- Csak a célzott mező változik, a többi érintetlen marad
- reset() – visszaállítás az eredeti értékekre (jelszómezők ürítése, imageFile null, boolean flagek false)

- Hiányzó/üres user mezők graceful kezelése

9.1.4 useRecipeFilter – Recept szűrő hook

Összetett szűrő logika tesztje egy makeRecipe helper factory segítségével. Szűrési dimenziók:

- Szöveges keresés (name, case-insensitive)
- Tag-ek: isVegan, isVegetarian, isCommunity (null = nem szűr)
- Kategória (egy vagy több egyszerre)
- Kalória tartomány (min/max, részleges bound)
- Elkészítési és főzési idő (max bound)
- Adagszám (pontos egyezés)
- resetFilters() – visszaállítás alapállapotra
- Kombinált szűrők együttes alkalmazása

9.1.5 validateRecipePayload – Recept payload validáció

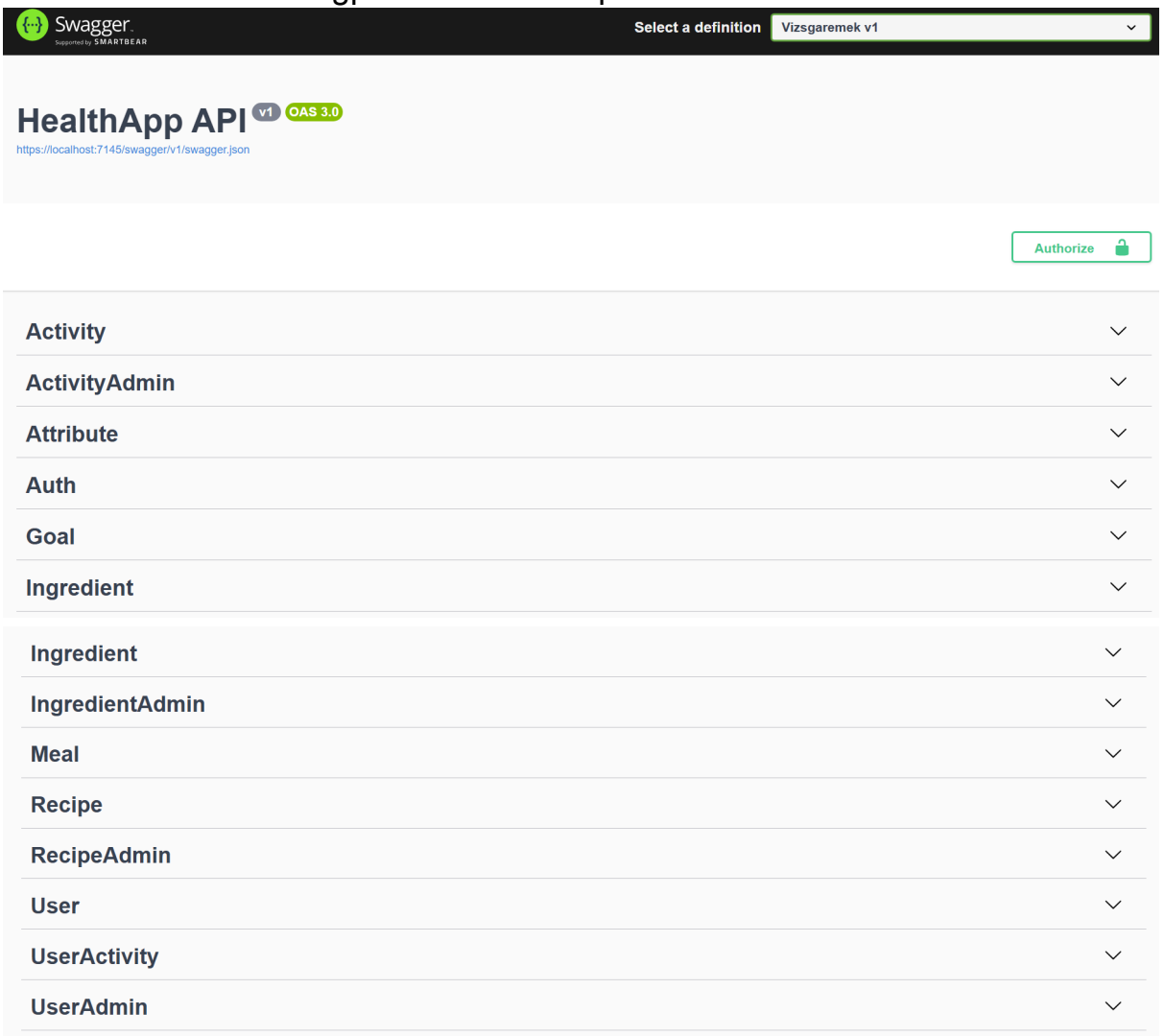
Egységtesztek: null-t ad vissza sikeres validáció esetén, string hibaüzenetet hiba esetén. Az ellenőrzött mezők: name, description, portions, preparationTime, cookingTime, instructions, ingredients (tömb üresség + ingredientId/amount > 0), kalória/protein/szénhidrát/zsír, és opcionális kép.

9.2 Backend tesztelése

9.2.2 Swagger UI

A backend végpontjainak tesztelése a beépített Swagger UI felületen keresztül történt, amely a backend indítása után automatikusan elérhető a https://localhost:7145/swagger/index.html címen. A Swagger felület lehetővé

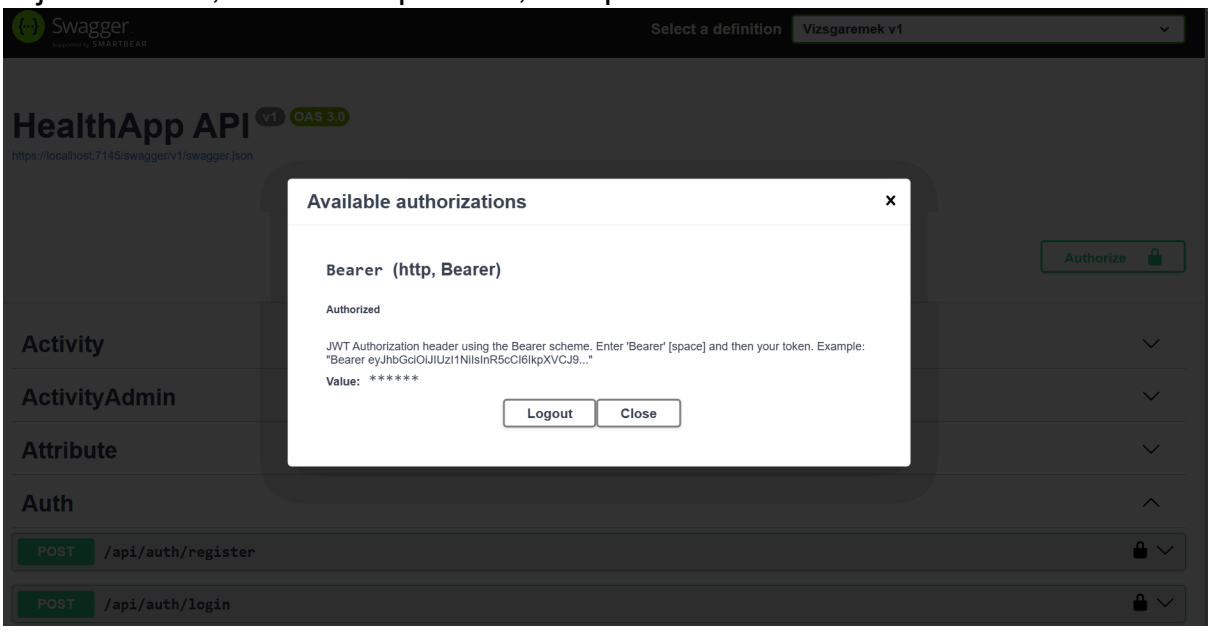
teszi az összes API végpont közvetlen kipróbálását.



9.2.2 Bejelentkezést igénylő végpontok tesztelése

A JWT token megadásával hitelesített kérések is küldhetők az "Authorize" gombra kattintva. A tesztelés során ellenőrzésre kerültek a regisztráció,

bejelentkezés, étkezés naplózása, receptkezelés és az admin műveletek.



10. Továbbfejlesztési lehetőségek

10.1 Rövidtávú továbbfejlesztési lehetőségek

10.1.1 Bevásárlólista generálása

Amikor a felhasználónak van egy heti étkezési terve, ki tudja exportálni az összes szükséges hozzávalót egy listába. A rendszer összesítené a receptek alapanyagait, összevonná az ismétlődőket, és kategóriánként csoportosítaná őket (tejtermékek, zöldségek, stb.). Akár egy interaktív checklist formájában is megjelenhetne ahol ki lehet pipálni a már megvett dolgokat.

10.1.2 Kalóriakölcsönzés

Ha a felhasználó egy nap túleszik, a következő nap ajánlott kalóriája automatikusan csökkenne a különbséggel kompenzálva. Ez egy egyedi megkülönböztető feature lenne az alkalmazásban.

10.1.3 Közösségi funkciók (Kommentek, kedvelések, értékelések)

A felhasználók kedvelhetnének recepteket és követhetnének más felhasználókat akiknek a tartalmát szeretik. A recept oldalán megjelenhetne egy komment szekció, és egy egyszerű "követett felhasználók" feed

megjelenítheti az általuk feltöltött új recepteket. Ezen kívül 1-5 csillagos értékelés kerülhet a közösségi receptekre, ami befolyásolja hogy azok hogyan jelennek meg a heti étkezési terv generátorban. A magasabban értékelt receptek gyakrabban kerülnének ajánlásra.

10.2 Hosszabb távú továbbfejlesztési lehetőségek

10.2.1 Gamifikáció

Egy pont és achievement rendszer ahol a felhasználók jelvényeket kapnak például 7 egymást követő nap étkezés naplózásáért, a kalória cél teljesítéséért, egy heti terv elvégzéséért, vagy 10 különböző recept kipróbálásáért. A dashboardon egy "streak" számláló jó vizuális motivátor lenne.

10.2.2 Értesítések, emlékeztetők

Étkezés naplózási emlékeztetők, heti terv generálási értesítések, vagy cél határidő figyelmeztetések böngésző push notification formájában.

11. Irodalomjegyzék

1. React dokumentáció – https://react.dev
2. TypeScript kézikönyv – https://www.typescriptlang.org/docs/
3. Vite dokumentáció – https://vitejs.dev/guide/
4. Tailwind CSS dokumentáció – https://tailwindcss.com/docs
5. React Router dokumentáció – https://reactrouter.com/en/main
6. Axios dokumentáció – https://axios-http.com/docs/intro
7. Framer Motion dokumentáció – https://www.framer.com/motion/
8. Material UI dokumentáció – https://mui.com/material-ui/getting-started/
9. Headless UI dokumentáció – https://headlessui.com
10. Vitest dokumentáció – https://vitest.dev/guide/
11. React Testing Library dokumentáció – https://testing-library.com/docs/react-testing-library/intro/
12. Lucide React ikonkönyvtár – https://lucide.dev/guide/packages/lucide-react
13. React Three Fiber dokumentáció – https://docs.pmnd.rs/react-three-fiber/getting-started/introduction
14. JWT Authentication best practices – https://auth0.com/docs/secure/tokens/json-web-tokens

15. Stackoverflow – https://stackoverflow.com/questions
16. Microsoft C# documentation – https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/